

YANAX

CLIENT PORTAL

Especificación de Ingeniería de Software

Versión: 1.0 Estado: Propuesta para MVP Fecha: 27 de septiembre de 2026

Portal web para seguimiento, documentación y comunicación de proyectos de software entre Yanax Studio y sus clientes.

Documento	Contenido
Objetivo	Definir qué se construirá y bajo qué condiciones.
Audiencia	Yanax Studio, desarrollo, diseño, QA y revisión externa con Claude.
Alcance	MVP de portal multi-proyecto con aislamiento por cliente.
Stack base	React + Vite + TypeScript + Tailwind + Supabase + Vercel

Principio del producto: reducir las consultas repetitivas de estado sin reducir la comunicación humana cuando realmente sea necesaria.

1. Resumen ejecutivo

Yanax Client Portal es una aplicación web que permitirá a los clientes consultar de forma autónoma el avance de sus proyectos de software. El sistema centralizará sprints, tareas, actualizaciones, imágenes, videos, documentos y comentarios.

El producto nace de un problema operativo concreto: el equipo de Yanax recibe preguntas recurrentes sobre el estado del proyecto y necesita invertir tiempo en responderlas manualmente. El portal convierte esa información en una fuente de consulta permanente, estructurada y trazable.

El MVP será una aplicación web responsive. No se desarrollará una aplicación nativa para iOS/Android en esta etapa. El acceso se realizará mediante navegador y URL.

La solución utilizará Supabase como plataforma backend, aprovechando PostgreSQL, Auth, Storage y Row Level Security (RLS). Esto reduce la cantidad de infraestructura que Yanax deberá mantener.

2. Problema y oportunidad

Problema principal: los clientes necesitan visibilidad del progreso, pero la información puede estar distribuida entre WhatsApp, archivos, mensajes y conversaciones.

Consecuencias: interrupciones al equipo, respuestas repetitivas, dificultad para reconstruir el historial, poca trazabilidad y una experiencia de seguimiento dependiente de una persona.

Oportunidad: convertir el seguimiento en un producto sistemático de Yanax. El cliente puede consultar el estado cuando quiera y Yanax conserva un historial organizado de lo que se entregó y comunicó.

3. Objetivos del sistema

3.1 Objetivo general

Proporcionar un portal web seguro donde cada cliente pueda consultar exclusivamente la información de sus proyectos y comunicarse con Yanax mediante comentarios o un canal directo de WhatsApp.

3.2 Objetivos específicos

- Reducir preguntas repetitivas sobre el estado del proyecto.
- Centralizar la información de avance por sprint.
- Documentar visualmente el desarrollo mediante imágenes y videos.
- Permitir comentarios asociados al proyecto.
- Proporcionar al administrador herramientas CRUD para gestionar clientes, proyectos, sprints, tareas, actualizaciones y archivos.
- Garantizar aislamiento de datos entre clientes mediante autorización y RLS.
- Mantener una experiencia responsive desde computador y teléfono.

4. Alcance del MVP

Área	Incluido en MVP	Fuera del MVP
Autenticación	Login, logout, recuperación de contraseña	SSO, MFA obligatorio
Clientes	CRUD administrativo	Portal de facturación

Área	Incluido en MVP	Fuera del MVP
Proyectos	CRUD, estado, fechas, progreso	Gestión financiera
Sprints	CRUD, estados, fechas, descripción	Planificación avanzada automática
Tareas	CRUD y estados	Asignación granular a desarrolladores
Actualizaciones	Publicación de novedades	Generación automática por IA
Media	Imágenes, videos y documentos	Edición de video
Comentarios	Cliente ↔ Yanax	Chat en tiempo real completo
Contacto	Botón WhatsApp	Integración oficial WhatsApp API
Analítica	Actividad básica	BI y métricas avanzadas

5. Actores y roles

Actor	Descripción	Permisos principales
Administrador Yanax	Gestiona la plataforma y todos los proyectos	Acceso total al ámbito administrativo
Cliente	Usuario externo asociado a uno o más proyectos	Solo lectura/comentarios sobre proyectos autorizados

Nota: la autorización se debe aplicar tanto en la interfaz como en la capa de datos. Ocultar una pantalla no constituye una medida de seguridad.

6. Requisitos funcionales

ID	Requisito	Descripción
RF-01	Autenticación	El sistema deberá permitir iniciar y cerrar sesión de forma segura.
RF-02	Recuperación	El usuario deberá poder recuperar su contraseña mediante el mecanismo de autenticación configurado.
RF-03	Gestión de clientes	El administrador podrá crear, consultar, actualizar y desactivar clientes.
RF-04	Gestión de usuarios	El administrador podrá crear/invitar usuarios y asociarlos con proyectos.
RF-05	Gestión de proyectos	El administrador podrá crear, editar, consultar, archivar y eliminar proyectos según las reglas de negocio.
RF-06	Acceso por proyecto	Un cliente solo podrá consultar proyectos a los que esté explícitamente asociado.
RF-07	Gestión de sprints	El administrador podrá crear, editar, cambiar estado y ordenar sprints.
RF-08	Gestión de tareas	El administrador podrá crear, actualizar y cambiar el estado de tareas.
RF-09	Actualizaciones	El administrador podrá publicar actualizaciones vinculadas a un sprint.
RF-10	Multimedia	El administrador podrá cargar imágenes, videos y documentos vinculados a un proyecto o sprint.
RF-11	Consulta multimedia	El cliente podrá visualizar o descargar archivos según el tipo y permiso configurado.
RF-12	Comentarios	El cliente podrá publicar comentarios sobre un proyecto; Yanax podrá responder.

ID	Requisito	Descripción
RF-13	WhatsApp	El portal deberá ofrecer un enlace directo al número de WhatsApp definido por Yanax.
RF-14	Dashboard	El cliente verá estado, progreso, sprint actual y actividad reciente de sus proyectos.
RF-15	Auditoría básica	El sistema deberá conservar fecha, usuario y acción de eventos relevantes de administración.

7. Requisitos no funcionales

ID	Categoría	Criterio
RNF-01	Seguridad	Aplicar RLS en Supabase y autorización por rol/proyecto.
RNF-02	Privacidad	Los datos de un cliente no deben ser accesibles a otro cliente.
RNF-03	Responsive	La interfaz debe funcionar correctamente en móvil, tablet y escritorio.
RNF-04	Rendimiento	Las vistas comunes deben cargar de forma ágil; paginar listas y evitar consultas innecesarias.
RNF-05	Disponibilidad	Usar infraestructura gestionada para reducir mantenimiento operativo.
RNF-06	Usabilidad	Un cliente debe identificar el estado del proyecto y la última actualización sin navegar por múltiples pantallas.
RNF-07	Mantenibilidad	Usar TypeScript, componentes reutilizables, validación y estructura modular.
RNF-08	Escalabilidad	La arquitectura debe permitir agregar más proyectos/clientes sin rediseñar el dominio.
RNF-09	Accesibilidad	Contraste adecuado, navegación por teclado en elementos relevantes y etiquetas claras.
RNF-10	Observabilidad	Registrar errores de frontend y eventos administrativos críticos.

8. Casos de uso principales

ID	Caso de uso	Actor	Resultado
CU-01	Iniciar sesión	Cliente/Admin	Autenticarse y acceder al área correspondiente.
CU-02	Consultar proyecto	Cliente	Ver información, progreso, sprint actual y actividad.
CU-03	Consultar sprint	Cliente	Ver descripción, tareas, actualizaciones y evidencias.
CU-04	Comentar	Cliente	Enviar una observación sobre el proyecto.
CU-05	Responder comentario	Admin	Responder una observación del cliente.
CU-06	Gestionar proyecto	Admin	Crear, editar, archivar y consultar proyectos.
CU-07	Gestionar sprint	Admin	Crear y actualizar sprints.
CU-08	Publicar avance	Admin	Publicar actualización de estado.
CU-09	Subir evidencia	Admin	Subir imágenes, videos o documentos.
CU-10	Gestionar acceso	Admin	Asociar usuarios clientes a proyectos.

9. Reglas de negocio

- 1 RN-01: un cliente solo puede consultar proyectos asociados a su usuario.
- 2 RN-02: un proyecto pertenece a un cliente/organización dentro del MVP, aunque el modelo deberá permitir múltiples miembros.
- 3 RN-03: un sprint pertenece a un proyecto.
- 4 RN-04: una tarea pertenece a un sprint.
- 5 RN-05: una actualización debe estar asociada a un sprint o proyecto según el contexto.
- 6 RN-06: los archivos deben tener propietario lógico (proyecto/sprint) y metadatos de tipo.
- 7 RN-07: el porcentaje de progreso debe tener una única fuente de cálculo definida; para el MVP se recomienda calcularlo a partir de tareas completadas en lugar de permitir porcentajes arbitrarios.
- 8 RN-08: eliminar físicamente información crítica debe estar restringido; se recomienda archivado/desactivación para clientes y proyectos.
- 9 RN-09: comentarios deben conservar autor y fecha.
- 10 RN-10: cualquier acceso a datos debe pasar por las políticas de autorización de Supabase.

10. Arquitectura recomendada

Para el MVP se recomienda una arquitectura frontend + backend gestionado. No se necesita inicialmente un servidor FastAPI/Node independiente.

Arquitectura lógica:

Navegador → React/Vite → Supabase SDK → Auth / PostgreSQL / Storage

Componentes:

- Frontend: React + Vite + TypeScript.
- UI: Tailwind CSS + shadcn/ui + Lucide.

- Estado remoto: TanStack Query si la complejidad de consultas lo justifica; para un MVP pequeño puede iniciarse con hooks de Supabase.
- Backend: Supabase.
- DB: PostgreSQL administrado por Supabase.
- Auth: Supabase Auth.
- Storage: Supabase Storage.
- Autorización: PostgreSQL RLS.
- Hosting: Vercel.
- Errores: Sentry o alternativa equivalente cuando el MVP pase a producción.

Cuándo introducir FastAPI/Node: si aparecen integraciones externas, procesamiento pesado, webhooks complejos, generación de documentos, jobs o lógica que no convenga ejecutar directamente desde el cliente. No se recomienda añadirlo solo por tener una arquitectura más compleja.

11. Arquitectura de frontend

Estructura sugerida:

src/ app/ components/ features/ auth/ dashboard/ projects/ sprints/ tasks/ comments/ media/ admin/ lib/ supabase/ validation/ routes/ types/ hooks/ layouts/

Principios:

- Separar componentes de UI de lógica de dominio.
- Centralizar tipos TypeScript generados desde la base de datos.
- Validar formularios antes de enviar.
- No guardar secretos de Supabase en el frontend; solo usar la clave pública/anon correspondiente y confiar en RLS.
- Evitar duplicar consultas y lógica de permisos en múltiples componentes.

12. Modelo de datos recomendado

Entidades principales:

Tabla	Propósito	Relaciones principales
profiles	Perfil extendido del usuario autenticado	auth.users
clients	Organización/cliente	projects, client_users
client_users	Usuarios pertenecientes a un cliente	clients, profiles
projects	Proyecto de software	clients, project_members, sprints
project_members	Acceso de usuarios a proyectos	projects, profiles
sprints	Unidad de trabajo	projects, tasks, updates
tasks	Trabajo concreto del sprint	sprints
updates	Publicaciones de avance	sprints/projects, profiles

Tabla	Propósito	Relaciones principales
media	Metadatos de archivos	projects/sprints, profiles
comments	Conversación sobre proyecto	projects, profiles
audit_logs	Trazabilidad administrativa	profiles

Relación conceptual:

Client → Projects → Sprints → Tasks / Updates / Media; Profiles → Project Members / Comments / Updates / Audit Logs.

13. Esquema inicial de PostgreSQL

profiles - id UUID PK → auth.users.id - full_name TEXT - role ENUM('admin','client') - avatar_url TEXT NULL - created_at TIMESTAMPTZ clients - id UUID PK - company_name TEXT - contact_name TEXT - phone TEXT NULL - status ENUM('active','inactive') - created_at TIMESTAMPTZ client_users - client_id UUID FK - user_id UUID FK - PRIMARY KEY(client_id,user_id) projects - id UUID PK - client_id UUID FK - name TEXT - description TEXT - status ENUM('planning','in_progress','paused','completed','archived') - start_date DATE - estimated_end DATE NULL - created_at TIMESTAMPTZ project_members - project_id UUID FK - user_id UUID FK - PRIMARY KEY(project_id,user_id) sprints - id UUID PK - project_id UUID FK - name TEXT - description TEXT - status ENUM('pending','in_progress','completed') - position INT - start_date DATE NULL - end_date DATE NULL tasks - id UUID PK - sprint_id UUID FK - title TEXT - description TEXT NULL - status ENUM('pending','in_progress','completed') - priority ENUM('low','medium','high') - position INT updates - id UUID PK - project_id UUID FK - sprint_id UUID FK - author_id UUID FK - title TEXT - content TEXT - created_at TIMESTAMPTZ media - id UUID PK - project_id UUID FK - sprint_id UUID FK - uploaded_by UUID FK - storage_path TEXT - file_name TEXT - mime_type TEXT - size_bytes BIGINT - created_at TIMESTAMPTZ comments - id UUID PK - project_id UUID FK - author_id UUID FK - content TEXT - created_at TIMESTAMPTZ - updated_at TIMESTAMPTZ NULL audit_logs - id UUID PK - actor_id UUID FK - action TEXT - entity_type TEXT - entity_id UUID - metadata JSONB - created_at TIMESTAMPTZ

14. Seguridad y autorización

La seguridad es un requisito de primer nivel porque el sistema es multi-cliente. La regla fundamental es: un cliente nunca debe poder consultar datos de otro cliente, incluso si conoce un ID o modifica una URL.

Controles recomendados:

- Supabase Auth para identidad.
- RLS en todas las tablas expuestas.
- Políticas basadas en auth.uid().
- Validación de pertenencia a client/project antes de SELECT, INSERT, UPDATE o DELETE.
- Buckets privados para documentos sensibles.
- URLs firmadas para archivos privados cuando sea necesario.
- Límites de tamaño y tipos MIME para uploads.
- No exponer service_role key en frontend.
- Auditoría de operaciones administrativas relevantes.
- Soft delete/archivado para información que deba conservar historial.

Ejemplo conceptual de política: permitir SELECT de un proyecto únicamente cuando existe una relación entre auth.uid() y project_members para ese proyecto, o cuando el usuario tiene rol admin.

15. API y acceso a datos

En el MVP se recomienda utilizar el cliente oficial de Supabase desde React y funciones SQL/RPC únicamente cuando una operación necesite transacciones o lógica de base de datos específica.

Dominio	Operaciones principales
Auth	signIn, signOut, resetPassword
Projects	list, get, create, update, archive
Sprints	list, get, create, update, reorder
Tasks	list, create, update, changeStatus
Updates	list, create, update, delete
Media	list, upload, delete, signedUrl
Comments	list, create, update
Admin	clients, users, projectMembers, audit

16. Flujos principales

16.1 Cliente consulta avance

- 1 Cliente inicia sesión.
- 2 Supabase Auth valida identidad.
- 3 Frontend consulta proyectos accesibles.
- 4 RLS filtra automáticamente los proyectos permitidos.
- 5 Cliente selecciona un proyecto.
- 6 Se muestran progreso, sprint actual, última actualización y actividad.
- 7 Cliente puede entrar a un sprint para ver evidencias y comentarios.

16.2 Admin publica avance

- 1 Admin abre proyecto.
- 2 Selecciona sprint.
- 3 Crea actualización.
- 4 Opcionalmente sube imágenes/video/documentos.
- 5 Supabase guarda metadatos y archivos.
- 6 La actualización queda visible para usuarios autorizados.

16.3 Cliente comenta

- 1 Cliente abre proyecto.
- 2 Escribe comentario.
- 3 Frontend valida contenido.
- 4 RLS comprueba que pertenece al proyecto.
- 5 Comentario se almacena con autor y timestamp.
- 6 Admin puede visualizarlo y responder.

17. Diseño UX/UI

El portal debe priorizar claridad sobre cantidad de información. El cliente debe poder entender el estado del proyecto en segundos.

Pantallas MVP:

- Login.
- Dashboard cliente.
- Detalle de proyecto.
- Detalle de sprint.
- Galería/documentación.
- Comentarios.
- Perfil.
- Dashboard administrativo.
- Gestión de clientes.
- Gestión de proyectos.
- Gestión de sprints/tareas.
- Gestión de archivos y actualizaciones.

Identidad visual recomendada: navy #172B36, teal #114C5A, orange #FF9932 y cream #F7F5F0. La interfaz debe sentirse B2B, limpia y confiable, no como una aplicación gamer o de consumo.

18. Criterios de aceptación del MVP

- Un cliente puede iniciar sesión y solo visualizar los proyectos autorizados.
- Un cliente no puede consultar por URL/ID un proyecto ajeno.
- El administrador puede crear un cliente y un proyecto.
- El administrador puede crear y completar sprints.
- El administrador puede registrar tareas y sus estados.
- El administrador puede publicar una actualización.
- El administrador puede subir al menos imágenes y documentos; video debe soportarse dentro de límites definidos.
- El cliente puede consultar las evidencias.
- El cliente puede comentar y el administrador puede responder.
- El botón de WhatsApp abre el número configurado por Yanax.
- Las operaciones críticas están protegidas mediante RLS.
- El sistema funciona en móvil y escritorio.
- El sistema no expone claves privadas ni service_role en el frontend.

19. Plan de desarrollo recomendado

Sprint	Objetivo	Entregables
Sprint 0	Fundación	Repositorio, Vite, TypeScript, Tailwind, Supabase, ambientes, linting, estructura.
Sprint 1	Auth + perfiles	Login, recuperación, roles, profiles, layouts.
Sprint 2	Clientes + proyectos	CRUD admin, project_members y dashboard básico.
Sprint 3	Sprints + tareas	CRUD, estados, orden, cálculo de progreso.
Sprint 4	Actualizaciones + media	Publicaciones, Storage, imágenes/documentos/video.
Sprint 5	Comentarios + contacto	Comentarios, respuestas, WhatsApp.
Sprint 6	Seguridad + QA	RLS, pruebas de autorización, validaciones, responsive.
Sprint 7	Producción	Deploy, dominio/subdominio, monitoreo, backup/operación.

20. Estrategia de pruebas

Pruebas mínimas:

- Unitarias para utilidades y lógica de transformación.
- Integración para operaciones críticas de Supabase.
- Pruebas de autorización con usuario cliente A intentando acceder a proyecto B.
- Pruebas de uploads y límites.
- Pruebas responsive en móvil y escritorio.
- Pruebas de formularios y validaciones.
- Pruebas de regresión antes de cada release.

Caso de seguridad obligatorio:

Crear Cliente A y Cliente B. Asociar Proyecto A a Cliente A. Iniciar sesión como usuario de Cliente B e intentar leer Proyecto A, Sprint A, comentario y media A. Todos deben ser rechazados por RLS.

21. Decisiones técnicas pendientes

Decisión	Recomendación inicial	Debe definirse antes de
Dominio	portal.yanax.dev o similar	Producción
Video	Supabase Storage para MVP pequeño; evaluar servicio externo si crece	Implementación de media
Correo	Supabase Auth + proveedor transaccional si se requieren avisos	Notificaciones
Progreso	Derivado de tareas completadas	Dashboard
Borrado	Archivar antes que borrar físicamente	CRUD admin

Decisión	Recomendación inicial	Debe definirse antes de
Realtime	No obligatorio en MVP	Comentarios/actividad
Notificaciones	No obligatorias en primera versión	Post-MVP

22. Riesgos

Riesgo	Impacto	Mitigación
RLS mal configurado	Alto	Pruebas explícitas de aislamiento y revisión de políticas.
Videos demasiado grandes	Medio	Límites de tamaño, compresión y evaluar storage externo.
MVP demasiado grande	Alto	No incluir facturación, IA, chat realtime o SaaS en V1.
Información duplicada	Medio	Modelo relacional claro y fuente única de verdad.
Dependencia excesiva de Supabase	Medio	Mantener dominio y lógica desacoplados; PostgreSQL estándar.
Cliente no revisa el portal	Medio	UX simple y notificaciones futuras cuando aporten valor.

23. Evolución posterior al MVP

- Notificaciones por email cuando se publica un nuevo avance.
- Aprobación formal de entregables.
- Cronograma/Gantt.
- Historial de cambios más completo.
- Roles adicionales: PM, desarrollador, diseñador, cliente.
- Integraciones con GitHub/GitLab para importar actividad técnica.
- Generación de reportes PDF.
- IA para resumir el sprint y redactar actualizaciones.
- Portal multi-tenant para convertir la solución en SaaS.
- Facturación y pagos si el producto deja de ser exclusivamente interno.

24. Definition of Done del MVP

El MVP se considerará terminado cuando:

- Todos los requisitos funcionales críticos estén implementados.
- Los clientes estén aislados mediante RLS.
- Los casos de acceso indebido hayan sido probados.
- El flujo completo Admin → Sprint → Tareas → Actualización → Media → Cliente sea funcional.
- El cliente pueda comentar y contactar por WhatsApp.
- La aplicación sea responsive.
- No existan secretos privados en el frontend.
- La aplicación esté desplegada en un entorno de producción.

- Existe documentación mínima para mantenimiento y despliegue.
- Al menos un proyecto piloto real pueda gestionarse de inicio a fin mediante el portal.

25. Resumen para transferencia a otro asistente

Este documento puede utilizarse como contexto base para continuar el diseño con otro modelo. La prioridad es no comenzar a programar antes de validar requisitos, modelo de datos, autorización y alcance del MVP. El siguiente artefacto recomendado es un paquete técnico que incluya ERD, SQL inicial de Supabase, políticas RLS, estructura de carpetas, contratos de datos y backlog detallado por sprint.

Decisión arquitectónica principal: React + Vite + TypeScript + Tailwind/shadcn en frontend; Supabase Auth + PostgreSQL + Storage + RLS como backend gestionado; Vercel como hosting. FastAPI/Node queda reservado para necesidades que realmente lo justifiquen.